GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 05 — Sockets TCP y UDP
Bloque 01

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea

Lo que veremos en este bloque



El socket
IP, puerto y protocolo



Cliente TCP
connect · sendall · recv



Servidor TCP
bind · listen · accept



Handshake
SYN · ACK · FIN · TIME_WAIT



Errores
Timeouts, excepciones y select

Todo con `socket`: la librería estándar de Python para redes. En el bloque 02: **UDP**, **HTTP y NTP** y la **práctica eco**.

Qué es un socket

La idea en una frase

> Un **socket** es el punto final de una conexión de red: la interfaz que tu programa usa para **enviar y recibir datos** a través de la red.

Piensa en una llamada telefónica: marcas, esperas a que contesten, habláis y colgáis. Tu teléfono es el **socket**; el número que marcas, la **IP**; y la extensión a la que pides hablar, el **puerto**. Sin ese trío no hay conversación posible.

ℹ️ Según cómo hable el socket, será **TCP** (la carta certificada) o **UDP** (el avión de papel). Esa elección marca toda la unidad.

IP y puerto

El «quién» y el «dónde»

Concepto	Qué identifica	Ejemplo
IP	La máquina en la red	`127.0.0.1` · `192.168.1.10`
Puerto	El programa dentro de la máquina	`80` (web) · `5000` · `9000`

🏠 `127.0.0.1` es **localhost**: tu propia máquina. Perfecto para pruebas sin red real.

🔢 Un puerto va del **0 al 65535**: el SO reserva los conocidos (80 web, 443 HTTPS); para tus programas usa **puertos altos** (5000, 9000).

ℹ️ La pareja **(IP, puerto)** se llama *dirección de socket* o *endpoint*, y en Python se pasa como una tupla: `("127.0.0.1", 5000)`.

Crear un socket en Python

Una línea que decide el protocolo

primer_socket.py — Python

```python
import socket
# Crear un socket TCP
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Parámetro	Significado
`AF_INET`	Familia de direcciones: **IPv4** (`AF_INET6` para IPv6)
`SOCK_STREAM`	Socket **TCP**: orientado a conexión, fiable
`SOCK_DGRAM`	Socket **UDP**: sin conexión, datagramas sueltos

✅ 💡 Truco para no liarte: **D**GRAM = datagramas sueltos (**UDP**); **S**TREAM = flujo continuo

La analogía del bloque

Carta certificada contra avión de papel

📜 TCP: la carta certificada

Pagas más, el cartero confirma la entrega y, si se pierde, **se reenvía**. Sabes que llegó… y en el orden correcto.

✈️ UDP: el avión de papel

Lo lanzas desde el balcón y lo olvidas. Si aterriza, bien; si el viento se lo lleva, también. Pero lanzas **cien** en lo que tardas una carta.

ℹ️ Esas dos filosofías definen el resto de la unidad: **fiabilidad contra velocidad**.

TCP vs UDP cara a cara

La tabla que hay que saberse

Característica	TCP	UDP
Conexión	Sí (handshake)	No
Entrega garantizada	Sí	No
Orden	Sí	No
Velocidad	Más lento	Más rápido
Uso típico	Web, correo, FTP	Streaming, juegos, DNS

ℹ️ Las tres filas de «No» son, paradójicamente, la razón de que **UDP sea tan rápido**: nada de confirmaciones ni reenvíos. Las hablaremos en el bloque 02.

Mini-chequeo: el socket

🧠 Responde en 30 segundos:

1. ¿Qué identifica la **IP** y qué identifica el **puerto**?
2. ¿Qué representa `127.0.0.1` y para qué sirve?
3. ¿Qué constante de Python crea un socket **TCP** y cuál uno **UDP**?
4. ¿Qué garantiza TCP que UDP no garantiza? (tres cosas)

🔄 **Respuestas:**

- La IP identifica la **máquina**; el puerto, el **programa** dentro de ella
- Es **localhost**: tu propia máquina, para pruebas sin red real
- `SOCK_STREAM` para TCP y `SOCK_DGRAM` para UDP, ambos con `AF_INET`
- **Conexión** (handshake), **entrega garantizada** y **orden**

Cliente TCP

La idea en una frase

> El cliente TCP **marca el número**: crea el socket, se conecta con `connect()`, envía con `sendall()`, recibe con `recv()` y cierra.

Es el lado que **inicia la conversación**: en la analogía del teléfono, eres tú marcando el número y esperando a que contesten. Sin cliente, el servidor esperaría en el puerto para siempre.

El ciclo de vida del cliente

Cuatro pasos, siempre en orden

1. `socket()` → **crear** el punto final de la conexión
2. `connect((HOST, PORT))` → **conectar**: dispara el three-way handshake
3. `sendall()` / `recv()` → **hablar**: enviar y recibir bytes
4. `close()` → **cerrar**. El `with` de Python lo hace solo al salir del bloque

El cliente TCP mínimo

Resuelto, de arriba a abajo

cliente.py — Python

```python
import socket
HOST, PORT = "127.0.0.1", 5000
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cliente:
    cliente.connect((HOST, PORT))        # 1. conectar
    cliente.sendall(b"Hola servidor!")   # 2. enviar
    respuesta = cliente.recv(1024)       # 3. recibir
    print(f"Respuesta: {respuesta.decode()}")
```

```
$ python cliente.py
Respuesta: Recibido: Hola servidor!
```

Cada método, al detalle

Método	Qué hace	¿Cuándo?
`socket(AF_INET, SOCK_STREAM)`	Crea el socket TCP	Siempre, lo primero
`connect((HOST, PORT))`	Establece la conexión (handshake)	Antes de hablar
`sendall(datos)`	Envía **todos** los bytes	Cuando quieras mandar
`recv(1024)`	Recibe hasta 1024 bytes	Cuando esperes respuesta
`close()` / `with`	Libera la conexión	Al terminar

⚠️ `connect()` lanza **ConnectionRefusedError** si no hay nadie escuchando en ese puerto. Y `recv(1024)` no es un límite absoluto: es el **tamaño máximo del buffer**; un mensaje mayor exige varios `recv()` y concatenar.

Be the code: el cliente, paso a paso

Sé el cliente. Cada paso, cada byte

1. `socket(AF_INET, SOCK_STREAM)` → nace el socket `cli`
2. `connect(("127.0.0.1", 5000))` → three-way handshake por la red
3. `sendall(b"Hola servidor!")` → los bytes salen hacia el servidor
4. `recv(1024)` → esperando respuesta... ⏳ (¡**bloqueante**!)
5. `print(...)` y `close()` → el `with` cuelga el teléfono

⚠️ `recv()` se queda esperando hasta que lleguen datos: si el servidor nunca responde, el programa queda colgado. La salida de emergencia son los **timeouts** (última parte de este bloque).

Mini-chequeo: cliente TCP

🧠 Responde sin mirar:

1. ¿Qué método establece la conexión y qué error lanza si no hay nadie escuchando?
2. ¿Por qué usar `sendall()` y no `send()`?
3. ¿Qué hace `recv(1024)` y por qué no es un límite absoluto?

🔄 **Respuestas:**

- `connect((HOST, PORT))`, que lanza **ConnectionRefusedError** sin servidor en ese puerto
- `sendall()` garantiza el envío de **todos los bytes**; `send()` puede mandar solo una parte
- Recibe **hasta 1024 bytes** por llamada: es el máximo del buffer; mensajes mayores requieren varios `recv()` concatenados

Servidor TCP

La idea en una frase

> El servidor TCP **descolgó primero**: reserva un puerto con `bind()`, avisa con `listen()` y se queda esperando con `accept()` a que un cliente llame.

Es la otra cara de la moneda del punto anterior: el que **escucha**. Sin `bind()` no tiene puerto; sin `listen()` no anuncia que espera llamadas; sin `accept()` no recibe a nadie.

El ciclo de vida del servidor

Seis pasos: descolgar y atender

1. `socket()` → crear el socket
2. `bind((HOST, PORT))` → **reservar** la dirección y el puerto
3. `listen()` → **anunciar** que espera conexiones entrantes
4. `accept()` → el «¿diga?»: espera a que suene el timbre (bloqueante)
5. `recv()` / `sendall()` → hablar con el cliente conectado
6. `close()` / `with` → cerrar la conversación

El servidor TCP mínimo

Con el with cerrando solo

servidor.py — Python

```python
import socket
HOST, PORT = "127.0.0.1", 5000   # localhost
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
    servidor.bind((HOST, PORT))              # 1. reservar puerto
    servidor.listen()                        # 2. anunciar escucha
    print(f"Servidor escuchando en {HOST}:{PORT}")
    conexion, direccion = servidor.accept()  # 3. esperar cliente (BLOQUEANTE)
    with conexion:
        datos = conexion.recv(1024)          # 4. recibir
        print(f"Recibido: {datos.decode()}")
        conexion.sendall(b"Recibido: " + datos)  # 5. responder
```

```
$ python servidor.py
Servidor escuchando en 127.0.0.1:5000
Conectado con ('127.0.0.1', 54321)
Recibido: Hola servidor!
```

Cada método del servidor, al detalle

Método	Qué hace	¿Cuándo?
`bind((HOST, PORT))`	Reserva la dirección y el puerto	Antes de escuchar
`listen()`	Prepara el socket para conexiones	Tras el bind
`accept()`	Espera un cliente: devuelve `(conexion, direccion)`	En bucle
`recv(1024)`	Recibe datos del cliente conectado	Cuando llegan
`sendall(datos)`	Envía la respuesta	Cuando hay que contestar

ℹ️ Fíjate: a partir de `accept()` todo se hace con **conexion**, no con **servidor**. El socket servidor solo escucha; `conexion` es el canal de esa llamada concreta.

El bucle de atención

Atender a más de un cliente

servidor_bucle.py — Python

```python
while True:
    conexion, direccion = servidor.accept()
    print(f"Conectado con {direccion}")
    with conexion:
        datos = conexion.recv(1024)
        print(f"Recibido: {datos.decode()}")
        conexion.sendall(b"Recibido: " + datos)
```

⚠️ Con este código básico el servidor atiende **un cliente cada vez**: si llega un segundo mientras el primero habla, se queda en la cola de `listen()`. Para atender varios a la vez hacen falta hilos o `select()`: eso es la **UD 06 · Servidores concurrentes**.

Pool puzzle: ordena el servidor

¿Formas un servidor TCP funcional?

desordenado.py — Python

```
a) with socket.socket() as srv:
b)     srv.listen()
c)     conn, addr = srv.accept()
d) import socket
e)     conn.sendall(b"Bienvenido\n")
f)     srv.bind(("127.0.0.1", 5000))
g)     with conn:
h)         datos = conn.recv(1024)
```

🔓 **Solución:** d → a → f → b → c → g → h → e

⚠️ **Errores típicos:**

- `listen()` antes de `bind()` → `OSError`
- `accept()` fuera del `with` → el socket se cierra
- `sendall()` antes de `recv()` → no es el protocolo típico (el cliente habla primero)

Mini-chequeo: servidor TCP

🧠 Responde en 30 segundos:

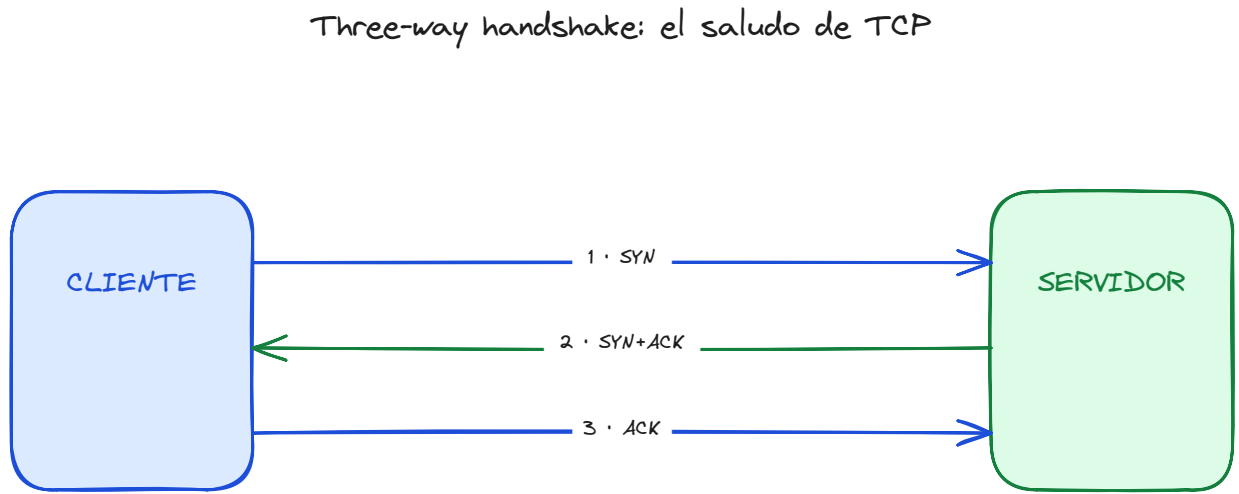
1. ¿Qué tres métodos preparan el servidor antes de atender a nadie?
2. ¿Qué devuelve `accept()` y por qué es bloqueante?
3. Tras `accept()`, ¿con qué objeto se recibe y envía: `servidor` o `conexion`?

🔄 **Respuestas:**

- `socket()`, `bind((HOST, PORT))` y `listen()`
- La tupla **(conexion, direccion)**: el canal de la conversación y la dirección del cliente. Bloqueante: espera a que un cliente llame
- Con **conexion**: `servidor` solo escucha; `conexion` es el canal de esa llamada

Three-way handshake

El saludo que abre toda conexión TCP



✔ Conexión establecida: los datos fluyen en ambas direcciones

En Python, todo esto ocurre dentro de connect()

El cierre de la conexión

La despedida: FIN y ACK

1. **FIN**: quien quiere cerrar avisa: «no tengo más que enviar»
2. **ACK**: el otro lado confirma y, cuando termina, envía su propio FIN
3. **ACK**: el primero confirma y la conexión se libera

ℹ️ En Python no gestionas el cierre a mano: el `with` (o `close()`) hace que el SO ejecute esta despedida. Pero **no es instantáneo**: el puerto queda retenido en estado **TIME_WAIT** durante un tiempo breve → siguiente diapositiva.

TIME_WAIT y «Address already in use»

El puerto que queda retenido

el_sorpresa.py — Python

```
# 1ª ejecución del servidor: todo bien
srv.bind(("127.0.0.1", 5000))   # ✅ OK
# Matas el servidor con Ctrl+C y lo relanzas al instante...
srv.bind(("127.0.0.1", 5000))   # 💥 OSError: Address already in use
```

⚠️ Tras el cierre, la conexión entra en **TIME_WAIT** (típicamente 1-2 min): el SO mantiene reservado el par (IP, puerto) para que los últimos FIN/ACK no queden huérfanos. Es **protección, no burocracia**... pero para quien reinicia su servidor mil veces al día, un estorbo.

SO_REUSEADDR

Reinicia tu servidor sin esperar

servidor_reutiliza.py — Python

```python
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("127.0.0.1", 5000))
servidor.listen()
```

Pieza	Significado
`SOL_SOCKET`	Nivel de opciones del propio socket
`SO_REUSEADDR`	La opción que permite reutilizar la dirección
`1`	Activa la opción (0 la desactivaría)

✅ El SO deja hacer `bind()` aunque queden conexiones en TIME_WAIT: mata y reinicia cuando quieras. **Pon esto siempre** en tus servidores: te ahorrará minutos de depuración.

Las redes fallan

try/except y timeouts: tu red de seguridad

conectar_seguro.py — Python

```python
import socket, time
def conectar_seguro(host, port, reintentos=3):
    for intento in range(reintentos):
        try:
            with socket.socket() as s:
                s.settimeout(5)
                s.connect((host, port))
                s.sendall(b"test")
                return s.recv(1024)
        except socket.timeout:
            print(f"⏱️ Timeout (intento {intento+1})")
        except ConnectionRefusedError:
            print("🚫 ¿El servidor está encendido?")
            time.sleep(1)
        except ConnectionResetError:
            print("💥 Cerraron la conexión de golpe")
        except OSError as e:
            print(f"❗ Error de red: {e}")
    return None
```

1. `settimeout(5)`: si algo tarda más de 5s, lanza `socket.timeout` en vez de bloquearse para siempre
2. Un `except` por excepción: cada fallo tiene su propio tratamiento
3. Reintenta hasta 3 veces; si no, `return None`: el llamador sabe que falló

El mapa de excepciones

Cada error, su tratamiento

Excepción	Cuándo ocurre
`socket.timeout`	La operación excede el tiempo límite
`ConnectionRefusedError`	No hay nadie escuchando en ese puerto
`ConnectionResetError`	El otro lado cerró de golpe
`ConnectionAbortedError`	El SO local aborta la conexión
`BrokenPipeError`	Escribes en un socket que ya se cerró
`OSError`	Red caída, DNS que no resuelve, etc.

ℹ️ 💡 Regla práctica: captura primero las **específicas** (timeout, ConnectionReset, BrokenPipe) y deja `OSError` como última red de seguridad.

Non-blocking y select

Tres formas de controlar la espera

esperas.py — Python

```python
import socket, select
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(5.0)      # Opción 1: timeout fijo
s.setblocking(False)   # Opción 2: no bloqueante (excepción si no hay datos)
# Opción 3: select, esperar con timeout en varios sockets
lectura, _, _ = select.select([s], [], [], 1.0)
if lectura:
    datos = s.recv(1024)
```

ℹ️ `select.select()` espera en **varios sockets a la vez sin hilos**: le pasas la lista y te avisa cuáles están listos. Lo usarás a fondo en la **UD 06 · Servidores concurrentes**.

Mini-chequeo: ciclo de vida y errores

🧠 Las cuatro de la entrevista:

1. ¿Cuáles son los tres mensajes del handshake y en qué orden?
2. ¿Qué error lanza el SO al relanzar un servidor sin `SO_REUSEADDR` y qué estado lo causa?
3. ¿Qué excepción lanza `recv()` si se pasa el timeout y cuál si escribes en un socket ya cerrado?
4. ¿Diferencia entre `ConnectionRefusedError` y `ConnectionResetError`?

🔄 **Respuestas:**

- **SYN** (cliente) → **SYN+ACK** (servidor) → **ACK** (cliente); luego fluyen los datos
- **OSError: Address already in use**, por conexiones en estado **TIME_WAIT**
- **socket.timeout** al exceder el tiempo; **BrokenPipeError** al escribir en un socket cerrado
- **Refused**: nadie escucha en el puerto (al conectar); **Reset**: el otro lado cerró de golpe (al hablar)

Resumen del bloque 01

Socket
Punto final; IP + puerto; STREAM = TCP

Cliente
socket → connect → sendall/recv → close

Servidor
bind → listen → accept → recv/send con *conexion*

Ciclo
SYN · SYN+ACK · ACK; FIN/ACK; TIME_WAIT → SO_REUSEADDR

Errores
settimeout, except específicos, select

✅ Bloque 02: **UDP** (sendto/recvfrom), **HTTP y NTP**, cuándo usar cada protocolo y la **práctica eco**.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //